OO Design:
Liskov Substitution Principle

Liskov Substitution Principle

- But we must be careful when we implement subclass to ensure that we don't violate the LSP

```
public class Rectangle {
   private double width, height;
   Rectangle(double w, double h) {
     width = w; height = h; }
   public double area() { return width * height; }
   public void setWidth(double w) { width = w;}
   public void setHeight(double h){ height = h;}
   public double getWidth() { return width; }
   public double getHeight() { return height;}
}
```

LSP Example

```
public class Square extends Rectangle {
  Square(double s) { super(s,s); }

   public void setWidth(double w) {
       super.setWidth(w); super.setHeight(w);}

   public void setHeight(double h){
       super.setWidth(h); super.setHeight(h);}

}
```

LSP Example

```
public class TestRectangle {
    public static void testLSP(Rectangle r) {
        r.setWidth(4.0);
        r.setHeight(5.0);
        System.out.println(r.area());
        assert(r.area()= 20);
    }
}
```

What Went Wrong?

- Consider the postcondition for setWidth in classes Rectangle and Square:
  - Rectangle: width == w && height == old.height
  - Square: width == w && height == w
- The postcondition for Square is weaker than the postcondition for Rectangle because it does not attempt to enforce the clause (height == old.height)

Rule

- When you override a method in a base class, the precondition of the overriding method should be *weaker* than the precondition of the overriden method:
  - The more useful a procedure is for clients
  - The more difficult it is to implement correctly

"Weaker" means the derived class should consider more inputs to the overriding method

Rule

- When you override a method in a base class, the postcondition of the overriding method should be *stronger* than the postcondition of the overriden method:

"Stronger" means the derived class should produce less outputs from the overriding method

So,

```
void testRectangleSetWidth(Rectangle rec)
{
    double oldHeight = rec.getHeight();
    rec.setWidth(5);
    // postconditions of Rectangle
    assertTrue (5 == rec.getWidth() &&
        oldHeight == rec.getHeight());
}
```

Subtype Substitution

- If *B* is a subtype of *A*, everywhere the code expects an *A*, a *B* can be used instead
- Examples:

```
Cell c = c1;
```

c1 must be a subtype of Cell (note *A* is a subtype of *A*)

```
Cell c = new ConwayLifeCell ();
```

~~ConwayLifeCell c = new Cell ();~~

Inheritance

- To implement a subtype, it is often useful to use the implementation of its supertype
- This is also called "subclassing"

```
class B extends A
```

B is a subtype of A

B inherits from A

both subtyping and inheritance

```
class C implements F
```

C is a subtype of F

just subtyping

No way to get inheritance without subtyping in Java

How do we know if saying *B* is a subtype of *A* is safe?

Substitution Principle: If *B* is a subtype of *A*, everywhere the code expects an *A*, a *B* can be used instead *and* the program still satisfies its specification

Subtype Condition 1: Signature Rule

We can use a subtype method where a supertype methods is expected:

- Subtype must implement all of the supertype methods
- Argument types must not be more restrictive
- Result type must be at least as restrictive
- Subtype method must not throw exceptions that are not subtypes of exceptions thrown by supertype

Signature Rule

```
class A {
   public R_A m (P_A p) ;
}
```

```
class B extends A {
   public R_B m (P_B p) ;
}
```

R_B must be a subtype of R_A: $R_B <= R_A$

P_B must be a ***super***type of P_A: $P_B >= P_A$